

CPT108 Assignment 2:
Weeks 1 to 6

Felix Lianto (2362727)

Instructor: Ho Lam

May 16, 2025

1 Problem 1

This problem will be broken up into 5 child problems, as per the problem statement. In each sub-problem, we will analyze its time complexity in terms of asymptotic notation according to each cases (worst-case, best-case, and average-case).

For our time complexity analysis, we will assume a uniform cost model, which indicates that every instruction will be assigned to some constant cost time [1], in other words, each instruction will take $O(1)$ time complexity. This will help simplify complexity analysis, as we would not have to worry about the size of numbers and can focus solely on the algorithm itself. Note that the approximation will be rough compared to the more precise cost model, the logarithmic cost model. That cost model will return a more precise approximation of the time complexity, at the cost of simplicity as it also considers the number of bits involved in an instruction [1].

Skipped Increments The following code snippet to divide n by two is given in Listing 1.

```
1      public void foo2(int n) {  
2          int s = 0;  
3          for (int i = 0; i < n; i += 2) {  
4              ++s;  
5          }  
6      }
```

Listing 1: Code provided.

From this snippet, we see that for line 2 and 4, the time complexity for that will be $O(1)$ as both will be executed with the same time regardless of the input n . However, line 3 declares a for-loop, but with an increment of two for each loop. Thus, the loop will end in roughly $\frac{n}{2}$ loops of the function, so the average-case complexity for this function is $O(n)$.

Moreover, the best-case and worst-case will follow a time complexity of $O(n)$, as there are no conditions where the loop exits early or some other conditional statements that will majorly affect the runtime no matter the integer input n .

Two For-loops Listing 2 shows the given code snippet used to multiply n by two.

```

1      public void foo3(int n) {
2          int s = 0;
3          for (int i = 0; i < n; ++i) {
4              ++s;
5          }
6          for (int j = 0; j < n; ++j) {
7              ++s;
8          }
9      }

```

Listing 2: Code provided.

Similar to the code in Listing 1, line 2, 4, and 7 all run on $O(1)$ complexity, as they do not depend on the input n . However, this function now has two for-loops, each runs for n loops. Since the only instruction that was looped was the increment instruction with $O(1)$ time complexity, as established before, this implies that for one for-loop, it will take $O(n)$ time to execute. Thus, for 2 loops, we will have $O(2n)$ time complexity, which is approximated to $O(n)$, meaning that the method runs in linear time for the average case.

By similar reasoning to the previous child problem, the best and worst-case time complexity also remains the same, $O(n)$, as the function still does not branch off or exit a loop early.

Power Function 1 The method in Listing 3 returns the result of x^n , where $x \in \mathbb{R}$ and $n \in \mathbb{Z}^+$.

```

1      public double power1(double x, int n) {
2          if (n == 0) {
3              return 1;
4          } else {
5              return x * power1(x, n-1);
6          }
7      }

```

Listing 3: Code provided.

Let us assume that the function calls and returning values all take $O(1)$ time. In the 5th line, it is given that the function will recurse itself by decrementing the value of n by 1. Thus, the function will call itself n times until it reaches 0, then it will start the multiplication process which

will take $O(1)$ time complexity. Thus, the total average time complexity for this implementation of the power function is $O(n)$.

Similarly, for the worst-case and best-case scenarios, the function will still have $O(n)$ time complexity, for similar reasoning to the previous sub-problems.

Power Function 2 The snippet in Listing 4 illustrates another implementation of the power function.

```
1      public double power2(double x, int n) {
2          if (n == 0) {
3              return 1;
4          } else if (n % 2 == 0) {
5              double y = power2(x, n/2);
6              return y * y;
7          } else {
8              double y = power2(x, n/2);
9              return x * y * y;
10         }
11     }
```

Listing 4: Code provided.

This function introduces the modulo operator, which we will assume will take $O(1)$ complexity, since $n \% 2$ is equivalent to $n \& 1$, where the ampersand is the bit-wise AND operator.

We will first focus on recursive aspect of this method first, namely in line 5 and 8. It is clear that the function will recurse until n has been completely divided and rounded down to 0. On average, this recursive calls will take $\log n$ to complete, given that the number divides by 2 each calls. Thus, for this function, it has an average time complexity of $O(\log n)$ thanks to its division approach.

Note that by the uniform cost model, the worst-case and best-case complexity also remains the same, $O(\log n)$. However, in the logarithmic cost model, the cases will have different time complexity depending on the exponent n . Specifically, the best case occurs when n is a power of two, as only multiplication by two numbers would be required, resulting in a time complexity of roughly $O(n^2 \log n)$. Meanwhile, the worst case occurs if the floor division of n keeps returning odd numbers, costing the machine running these instructions more as multiplication of three numbers are needed, totaling the time complexity of $O(n^4 \log n)$. This is assuming that the schoolbook long multiplication/division algorithm is utilized, however more precise and better implementations will be left as an exercise to the reader.

Power Function 3 The final implementation of the power function is given in Listing 5.

```

1      public double power3(double x, int n) {
2          if (n == 0) {
3              return 1;
4          } else if (n % 2 == 0) {
5              return power3(x, n/2) * power3(x, n/2);
6          } else {
7              return x * power3(x, n/2) * power3(x, n/2);
8          }
9      }

```

Listing 5: Code snippet.

This snippet is nearly identical to the one in Listing 4. However, it does not mean that the execution time is equal, and this is due to the repeated recursive calling of the function. The time complexity of each recursive call is $O(\log n)$ as it still divides the exponent n by two. However, the machine does not save the result of the first time it calculates the function, meaning it will continue to calculate and arrive at the same answer. Hence, while the time complexity across all 3 cases is still $O(\log n)$, as established before, this implementation will ultimately run slower as more redundant values need to be calculated by the machine.

2 Problem 2

Corollary 1. *Suppose that n is a power of two, that is, $n = 2^k$ for some non-negative $k \in \mathbb{Z}$. Then, the recurrence relation $T(n)$ defined in Equation 1 has asymptotic bounds in $\Theta(n \log^2 n)$, that is, $T(n) \in \Theta(n \log^2 n)$.*

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(\frac{n}{2}) + n \log n & n > 1 \end{cases} \quad (1)$$

Proof. Assume there exists some integer $k \geq 0$ such that $n = 2^k$. Then, by

repeated definition,

$$\begin{aligned}
T(2^k) &= 2T(2^{k-1}) + k \cdot 2^k \\
&= 2(2T(2^{k-2}) + (k-1) \cdot 2^{k-1}) + k \cdot 2^k \\
&= 4T(2^{k-2}) + (k-1) \cdot 2^k + k \cdot 2^k \\
&= 4T(2^{k-2}) + 2^k \cdot (k + (k-1)) \\
&= 4(2T(2^{k-3}) + (k-2) \cdot 2^{k-2}) + (k + (k-1)) \cdot 2^k \\
&= 8T(2^{k-3}) + (k-2) \cdot 2^k + k \cdot 2^k \\
&= 8T(2^{k-3}) + 2^k \cdot (k + (k-1) + (k-2)) \\
&\vdots \\
&= 2^x T(2^{k-x}) + 2^k \sum_{a=0}^{x-1} (k-a) && \text{(For some integer } 0 \leq x \leq k) \\
&\vdots \\
&= 2^k T(1) + 2^k \sum_{a=0}^{k-1} (k-a) \\
&= 2^k + 2^k \cdot (k + (k-1) + (k-2) + \dots + 3 + 2 + 1) && \text{(Expanded form)} \\
&= 2^k + 2^k \left(\frac{k(k+1)}{2} \right) && \text{(k-th Triangular number)} \\
T(2^k) &= 2^k \left(\frac{k^2 + k + 2}{2} \right).
\end{aligned}$$

Note that since $n = 2^k$, we also get the equivalent expression for $k = \log n$ where the logarithm is in base-2. Therefore, we can convert the closed-form equation from a function of 2^k to a function of n , as shown in Equation 2.

$$T(n) = n \left(\frac{\log^2 n + \log n + 2}{2} \right) \quad (2)$$

By multiplying the terms out, we see that there is a linear term n , a linearithmic term $\frac{n \log n}{2}$, and a quasilinear term $\frac{n \log^2 n}{2}$. Since as n goes larger to infinity, we see that $n \log^2 n$ will be the biggest contributor to the function. Hence, the recurrence relation $T(n)$ has the asymptotically tight bound of $\Theta(n \log^2 n)$. $\square$

3 Problem 3

Note that since $n = 2^{2^i}$ for some $i \geq 0$, we can define a function $f(i) = n = 2^{2^i}$, where $f(0) = 2$. Therefore, the recurrence relation $T(n)$ can be expanded

and then have its dependent variable changed to $f(i)$ as such;

$$\begin{aligned}
T(n) &= T(\sqrt{n}) + 1 \\
T(2^{2^i}) &= T(\sqrt{2^{2^i}}) + 1 \\
&= T(2^{\frac{2^i}{2}}) + 1 && \text{(Exponential definition of square root)} \\
&= T(2^{2^{i-1}}) + 1 \\
T(f(i)) &= T(f(i-1)) + 1 \\
&= T(f(i-2)) + 2 \\
&= T(f(i-3)) + 3 \\
&\vdots \\
&= T(f(i-x)) + x && \text{(For } 0 \leq x \leq i) \\
&\vdots \\
&= T(f(i-i)) + i \\
&= T(f(0)) + i \\
&= T(2) + i \\
T(f(i)) &= i + 1
\end{aligned}$$

Since $f(i) = n$ and $i = \log \log n$, we can convert this closed-form solution back to n in order to produce the solution in Equation 3.

$$\boxed{T(n) = 1 + \log \log n} \tag{3}$$

4 Problem 4

The following code snippet is given in Listing 6.

```

1      public static int foo_1(int[] nums, int left, int right) {
2          if (left == right) {
3              return left;
4          }
5
6          int mid = (left + right) / 2;
7          if (nums[mid] > nums[mid + 1]) {
8              return foo_1(nums, left, mid);
9          } else {
10             return foo_1(nums, mid + 1, right);
11         }
12     }
13
14     public static int foo(int[] nums) {
15         return foo_1(nums, 0, nums.length - 1);
16     }

```

Listing 6: Recursive code snippet.

From this, we will need to determine what the function does through an example input. So consider the following input array; [1, 3, 20, 4, 1, 0]. Then, the foo function will initialize the recursive function as shown in Figure 1.

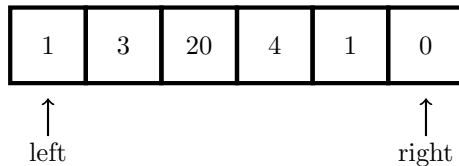


Figure 1: Initialization done by foo.

Then, the actual recursive function foo_1 starts very similarly to a binary search algorithm. It will calculate the middle index between the two index pointers, then compare the element at that index and the one to the right of it. If it is bigger, then the algorithm eliminates the right-half of the array, else it will eliminate the left-half of the array. Going back to our example, the function will narrow it down as shown in Figure 2.

Thus, we have arrived at the terminating condition, where the left index is equal to the right index. Therefore, the function will return 20 in this specific instance. In short, this function tries to **find a local maxima within a neighborhood of integers**, by searching from the center with binary search. Since the implementation is practically binary search, the time complexity of the function will be $O(\log n)$ in all 3 cases.

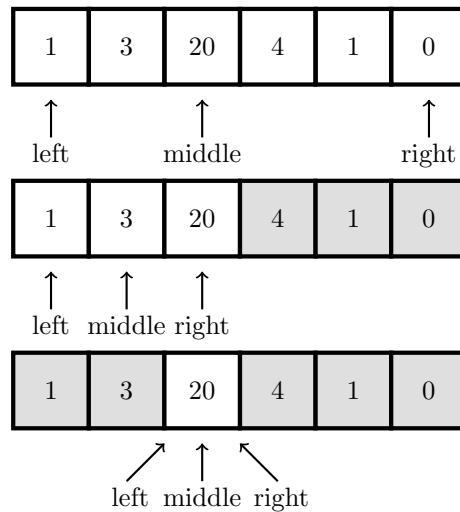


Figure 2: Recursive function walk-through.

5 Problem 5

We are given an array of 10 integers shown in Figure 3, and are asked to provide the array after the first iteration of the large loop, meaning the outer loop, of insertion sort. This implies that we need to place the 2nd element in the array to be sorted, so the sub-array from the 0th index to the 1st index is sorted. This process is illustrated in Figure 3.

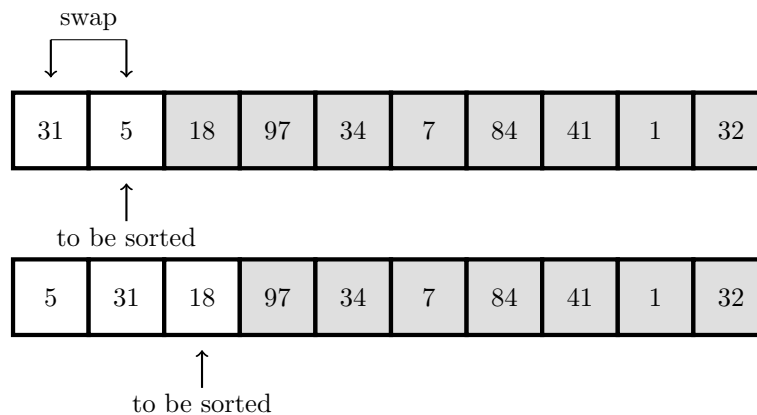


Figure 3: Array before and after the first large loop Insertion Sort.

6 Problem 6

Merge sort is a sorting algorithm that divides its input array into multiple sub-arrays and merges them while sorting in either ascending or descending order. An example of this algorithm in action is given in Figure 4.

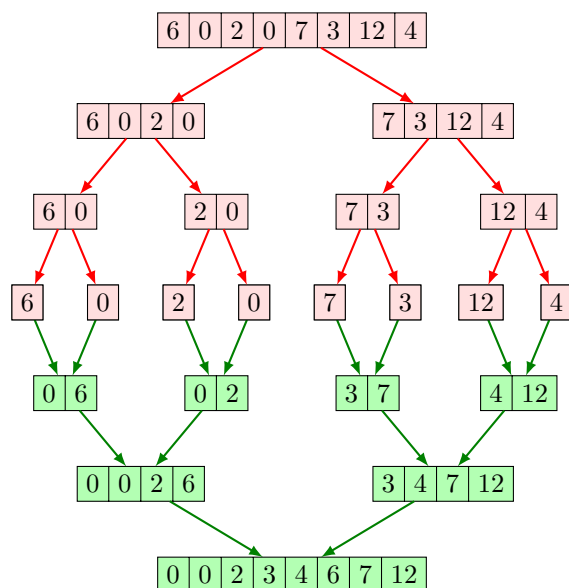


Figure 4: Merge Sort visualization.

7 Problem 7

To determine possible pivots after one round of partition in Quick Sort, we need to consider the elements to the left and right of the pivot candidate. That is, an element could be a pivot if all the elements to the left of it are smaller than the pivot element, and every element to the right are larger than the pivot. If one of these conditions is breached, then the element could not be a pivot. This can be quickly checked through the following function listed in Listing 7.

Thus, given an input of [12, 13, 4, 11, 7, 14, 26, 25, 23, 16, 15, 54, 75, 65, 59], we can either manually check each elements' left and right side or run the function in Listing 7 to arrive at the plausible pivot elements of [14 and 54].

8 Problem 8

We are given the following integer array; [24, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0], and are asked to perform the procedure MAX-HEAPIFY on the third element

```

1  public static List<Integer> possiblePivot(int[] A) {
2      List<Integer> result = new ArrayList<>();
3
4      int n = A.length;
5
6      pivotLoop:
7      for (int i = 0; i < n; i++) {
8          int currentElement = A[i];
9
10         // Check for elements from index 0 to index i-1.
11         for (int left = 0; left < i; left++) {
12             if (A[left] > currentElement) continue pivotLoop;
13         }
14
15         // Check for elements from index i to index n-1.
16         for (int right = i+1; right < n; right++) {
17             if (A[right] < currentElement) continue pivotLoop;
18         }
19
20         result.add(currentElement);
21     }
22
23     return result;
24 }

```

Listing 7: Java implementation to search for possible pivot elements.

as it is currently violating the max-heap property. This becomes more clear when we draw the heap diagram of this array, as displayed in Figure 5.

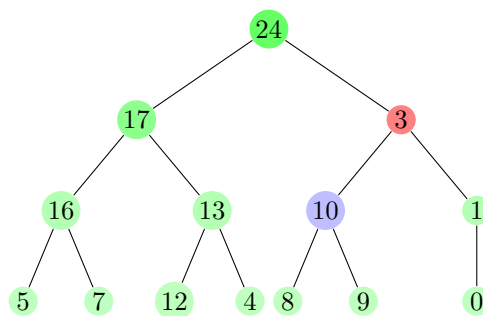


Figure 5: Current given heap representation of the array, with the violator highlighted in red and to be swapped with the element in blue.

From this, we can clearly see that the third element is violating the max-

heap property as the child node 10 is clearly bigger than it. Thus, to make sure this heap retains the max-heap property, we must exchange the parent-child node 3 and 10, as highlighted in Figure 5. However, we are still not done, as the property is still not met yet due to the next parent-children nodes 3, 8, and 9. Clearly, since 9 is the bigger one of the pair, we should exchange 3 with 9 to result in the final heap tree shown in Figure 6.

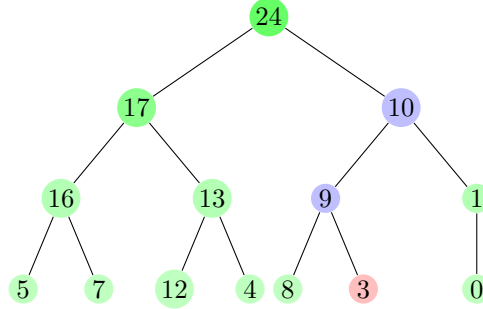


Figure 6: Max-heaped tree after the procedure, with the blue elements being elements that have been swapped and the red element being the violator.

Thus, the max-heaped integer array is given as follows;

[24, 17, 10, 16, 13, 9, 1, 5, 7, 12, 4, 8, 3, 0].

9 Problem 9

We are given an nested array, where the inner array has only 2 values. The problem states that sorting these arrays, we prioritize the first element, and in the case of a tie, we then look at the second value. Thus, to sort these arrays according to that criteria, it is more beneficial to work backwards, starting with the second value rather than the first. To sort the second value, since we don't care about the original order, we can utilize Quick Sort to sort these values. Then, to preserve the sorted second-value order that Quick Sort left, we can now sort it with a stable sorting algorithm such as Merge Sort. Hence, to sort these two-pair values, only the combination of **(C) Quick Sort first on the second values, followed by Merge Sort on the first values** will produce the correct result.

Since we are only running these sorting algorithms individually, the average-case and best-case time complexity will be $O(n \log n)$, assuming standard implementation. However, the worst-case time complexity will be $O(n^2)$ due to poor choice of pivots for Quick Sort. Since both sorting algorithms will take $O(n)$ space complexity, the overall algorithm will also take $O(n)$ space complexity too.

Bibliography

- [1] Juraj Hromkovič. *Theoretical Computer Science: Introduction to Automata, Computability, Complexity, Algorithmics, Randomization, Communication, and Cryptography*. Springer Science & Business Media, Sept. 18, 2003. 336 pp. ISBN: 978-3-540-14015-3.